Event ends in 2 days 2 hours 58 minutes.

In this exercise, you will set up your development environment with Kiro, install the AWS Documentation MCP server, set up the AWS architecture diagram agent skill, and create an automated agent hook for architecture diagram generation.

What You'll Learn

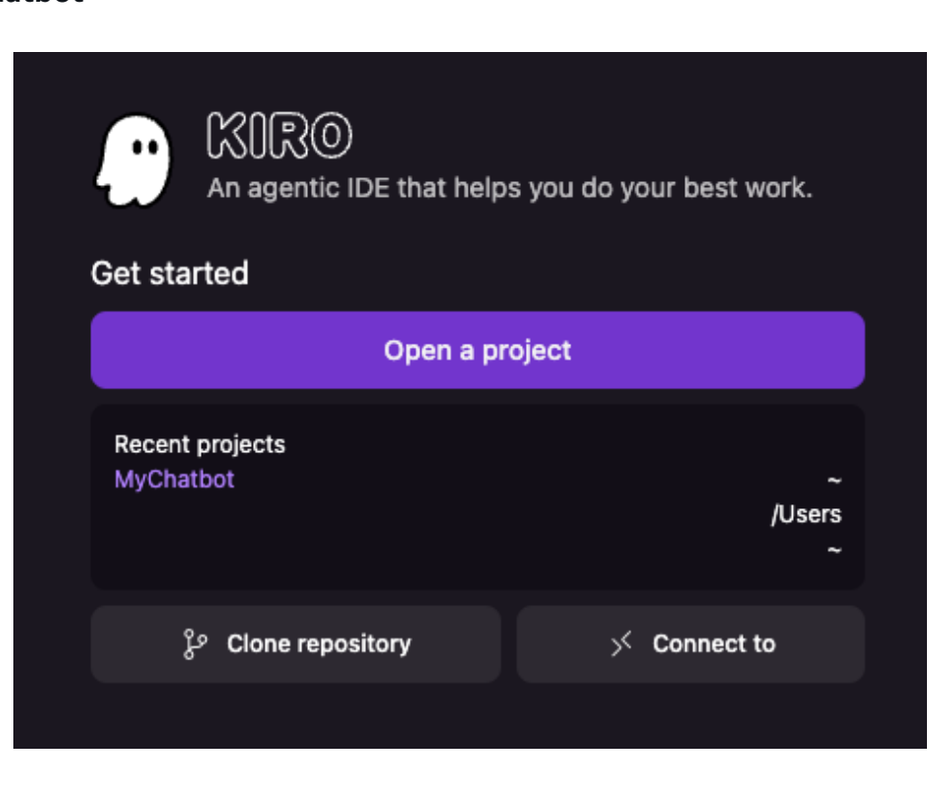
By the end of this exercise, you will have:

- Created a new Kiro project for chatbot development
- Installed the AWS Documentation MCP server
- Set up the AWS architecture diagram agent skill (via the deploy-on-aws agent plugin)
- Set up an agent hook for automatic architecture diagram generation
- Organized your project structure for the upcoming chatbot exercises

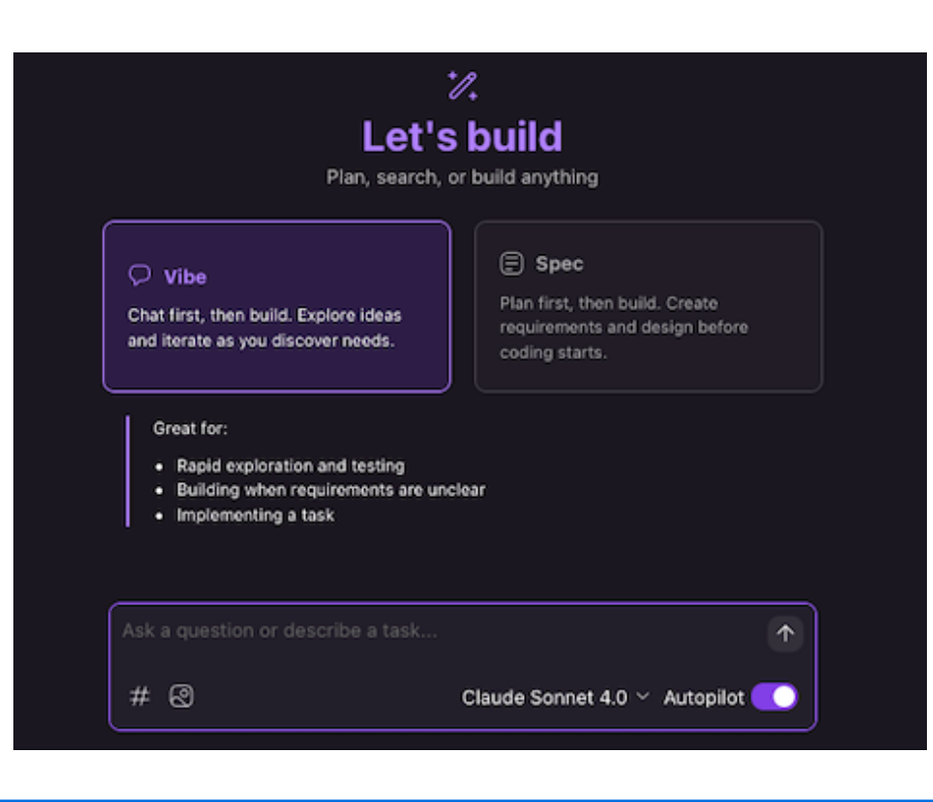
Step 1: Create Your Project

Create a new folder on your computer named **MyVibeChatbot** and open it in Kiro.

1. On your computer, create a folder named **MyVibeChatbot**



2. Open Kiro and click **Open a project**
3. Select the newly created **MyVibeChatbot** folder
4. In the "Let's build" page, select **Vibe** mode
5. Choose **Claude Sonnet 4.0** as your model



Vibe vs Spec Mode
For this exercise, we're using **Vibe** mode which allows for high-level, conversational development. Spec mode provides more structured, detailed development workflows.

Step 2: Install MCP Server

MCP (Model Context Protocol) is an open-source standard for connecting AI applications to external systems. MCP servers extend Kiro's capabilities with specialized tools. We'll install the AWS Documentation MCP server for this workshop:

- AWS Documentation MCP Server

Install `uv`

MCP servers for AWS tools are written in Python and managed via `uv` — a fast Python package manager built in Rust. You'll need to install `uv` before configuring the server.

In the Kiro chat area, type the following instruction:

```
Please help install 'uv' on my machine and make sure it's available in my PATH for the MCP server setup. After installed, please run these commands to confirm 'uv' i

uv --version
uvx --version
```

▸ Manually Installing 'uv' (If Issues Occur)

Install AWS Documentation MCP Server

Then, type the following instruction:

```
Please help me set up the AWS Documentation MCP server. I want to be able to access AWS service documentation directly in Kiro.
```

Let Kiro Handle Commands
When Kiro suggests running terminal commands, ask Kiro to execute them for you instead of running them manually. This ensures proper integration with your development environment.

Expected MCP Configuration

After installation, your MCP configuration should look similar to this:

▸ Expected MCP Servers Configuration - image

▸ Expected MCP Server Configuration - mcp.json

Step 3: Create Project Structure

Ask Kiro to set up the initial project structure for your chatbot:

```
Please create a well-organized folder structure for my chatbot project. I need:
- An architecture folder for diagrams and documentation
- A src folder for source code
- A docs folder for project documentation
- A README.md file explaining the project
```

Expected Project Structure

▸ Expected Folder Structure

Step 4: Add Workshop Steering Document

Steering documents are markdown files in `.kiro/steering/` that provide Kiro with project-specific context, constraints, and best practices. They act as guardrails that help Kiro make better decisions aligned with your project requirements throughout the workshop.

Think of it as a set of rules that Kiro automatically follows - like having an experienced developer looking over your shoulder, ensuring you stay within AWS workshop constraints and follow best practices.

Automatic Best Practices
With a steering document in place, Kiro will automatically:

- Follow AWS resource naming conventions (`kiro-workshop-` prefix)
- Use only allowed services (Lambda, S3, CloudFront, Bedrock, etc.)
- Apply required tags to all resources (`Project=kiro-workshop`)
- Prefer script reuse over manual commands to optimize token usage
- Verify CloudFormation templates against AWS documentation
- Use incremental stack updates instead of delete/recreate patterns

This means less time correcting mistakes and more time building your chatbot!

Steering Document Structure

▸ View Document Structure

Download and Install Steering Document

Download Steering Document

After downloading, ask Kiro to move the file to your project:

```
Please move the kiro-workshop-chatbot-steering.md file from my Downloads folder to .kiro/steering/ directory in my project.
```

Review Steering Document Contents

The steering document provides Kiro with essential constraints and patterns for this workshop. Here's what it contains:

▾ View Complete Steering Document Contents

Deployment | Requirements | Services | Architecture Patterns | Operations | Verification | Common Issues

Infrastructure Requirements:

- ALL infrastructure MUST use CloudFormation
- Region: us-east-1 (unless specified)

Environment Auto-Detection: Kiro automatically detects whether you're in Workshop Studio or using a personal AWS account:

```
CALLER_ARN=$(aws sts get-caller-identity --query 'Arn' --output text)
[[ "$CALLER_ARN" =~ WSParticipantRole|Participant ]] && WORKSHOP=true || WORKSHOP=false
```

Deployment Adjustments:

- **Workshop Studio:** Automatically adds `--role-arn` with CloudFormation service role
- **Personal Account:** Uses your default AWS credentials

Deploy command:

```
aws cloudformation deploy --template-file X.yaml --stack-name kiro-Y --capabilities CAPABILITY_NAMED_IAM
```

Automatic Inclusion
The steering document uses `inclusion\: auto` in its frontmatter, which means Kiro automatically loads it for all interactions. Other inclusion options available:

- `auto` - Always included automatically (used in this workshop)
- `manual` - Only included when explicitly referenced with `#` in chat
- `fileMatch` - Conditionally included when specific files are accessed

Verify Steering Document is Active

After moving the steering document, verify that Kiro has loaded it and understands the workshop constraints:

```
Based on the kiro-workshop-chatbot-steering.md steering document, what naming conventions should I use for AWS resources in this workshop? Also, what services am I a
```

Expected Response

Kiro should respond with specific workshop constraints from the steering document:

▸ Expected Kiro Response

Troubleshooting
If Kiro doesn't mention these specific constraints:

1. Verify the file is in `.kiro/steering/` directory (not in Downloads or static folder)
2. Check that the file has `inclusion: auto` in the frontmatter
3. Try asking: "Can you check if the kiro-workshop-chatbot-steering.md file exists in .kiro/steering/?"
4. Restart Kiro if needed to reload steering documents

Steering Document Active
Once verified, Kiro will automatically apply these workshop constraints to all your interactions, ensuring your chatbot follows AWS best practices and workshop requirements.

Step 5: Install the AWS Architecture Diagram Agent Skill

The AWS Diagram MCP server has been deprecated and replaced by the `aws-architecture-diagram` agent skill from the deploy-on-aws agent plugin. This skill generates validated draw.io architecture diagrams with official AWS4 icons — no separate MCP server required.

We'll install the `deploy-on-aws` plugin into Kiro using a conversion tool that transforms the plugin's skills into Kiro-compatible steering files.

Prerequisites

Paste the promp below to install Bun (a fast JavaScript runtime) and the `defusedxml` Python package:

```
Please help me install Bun on my machine. After that, also install the Python package defusedxml (version 0.7.1 or later) using pip3. Verify both are installed by ru

bun --version
pip3 show defusedxml
```

▸ Manually Installing Bun (If Issues Occur)

Install the deploy-on-aws Plugin for Kiro

In the Kiro chat area, type the following instruction:

```
Please run the following command to install the deploy-on-aws agent plugin (which includes the AWS architecture diagram skill) into this project:

COMPOUND_PLUGIN_GITHUB_SOURCE=https://github.com/awslabs/agent-plugins bunx @every-env/compound-plugin install deploy-on-aws --to kiro
```

This command downloads the `deploy-on-aws` plugin from the awslabs/agent-plugins repository and converts its skills and MCP server configs into Kiro's `.kiro/` format (steering files and MCP settings).

What Gets Installed
The conversion tool creates:

- Steering files in `.kiro/steering/` — containing the diagram generation skill instructions and reference docs
- MCP server configs in `.kiro/settings/mcp.json` — for the AWS Knowledge, AWS Pricing, and AWS IaC MCP servers used by the plugin

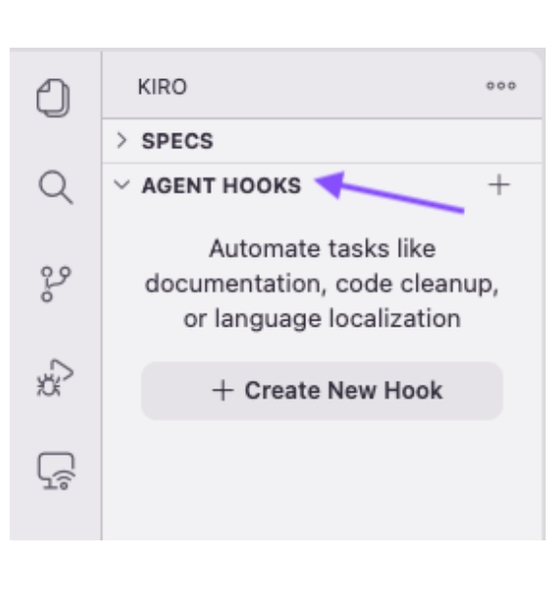
The `aws-architecture-diagram` skill enables Kiro to generate draw.io XML diagrams with official AWS4 icons, step legends, and dark mode support — all without a separate MCP server.

▸ Troubleshooting Plugin Installation

Step 6: Set Up Architecture Diagram Agent Hook

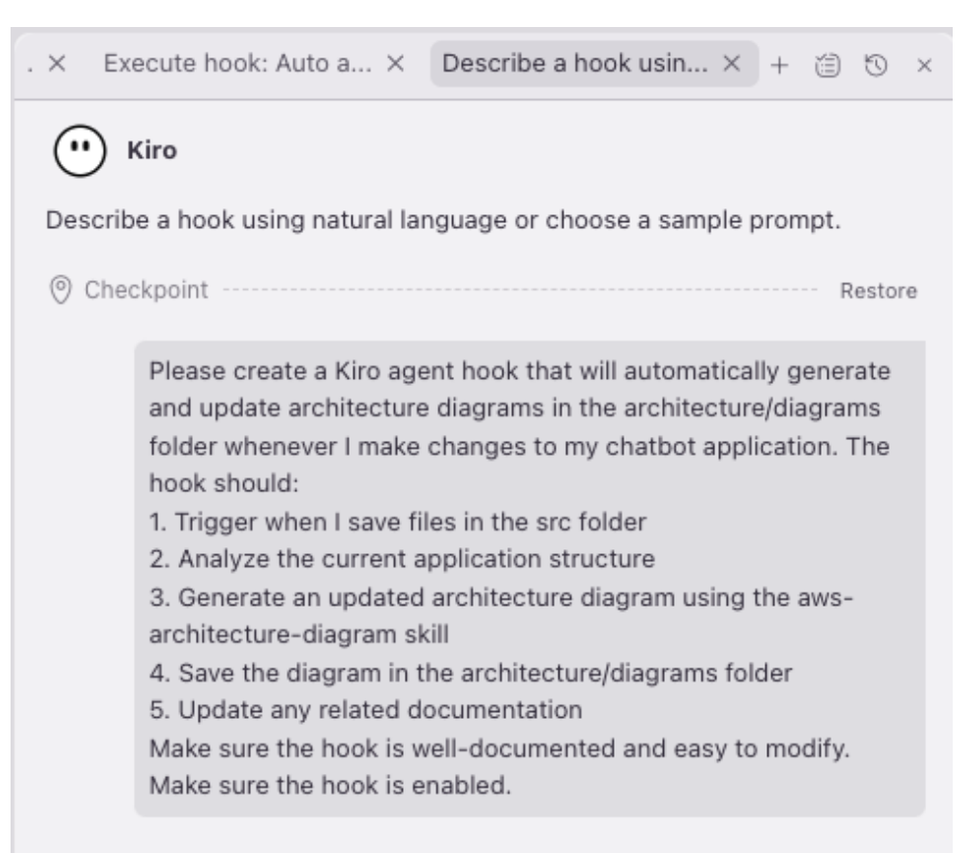
Now we'll create an automated agent hook that generates and updates architecture diagrams whenever you make significant changes to your chatbot.

Expand the "AGENT HOOKS" tab on the left panel.

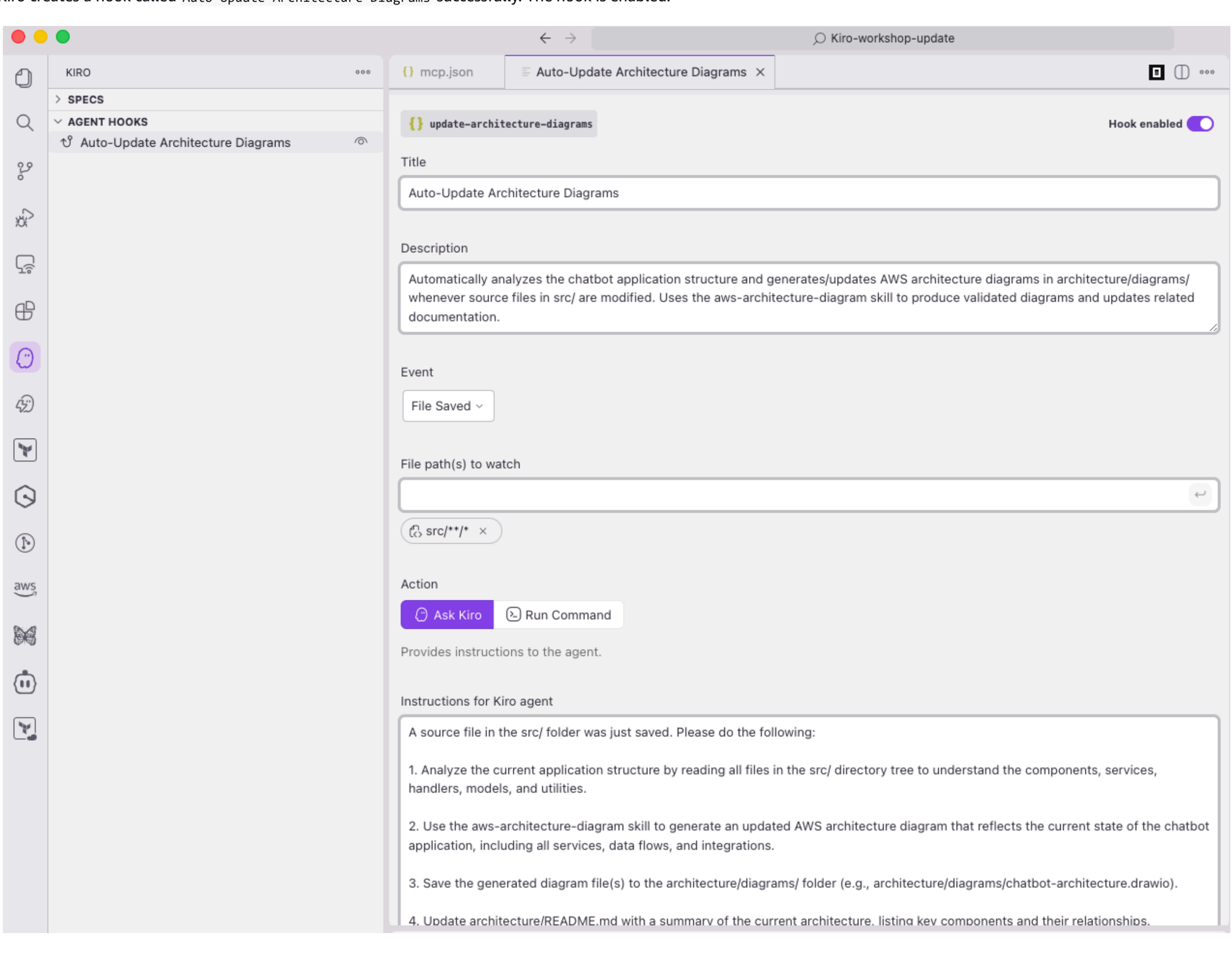


Click the `+ Create New Hook` button and select `Ask Kiro to create a hook`. Kiro will open a new Vibe chat session. Paste the text below into the chat and run.

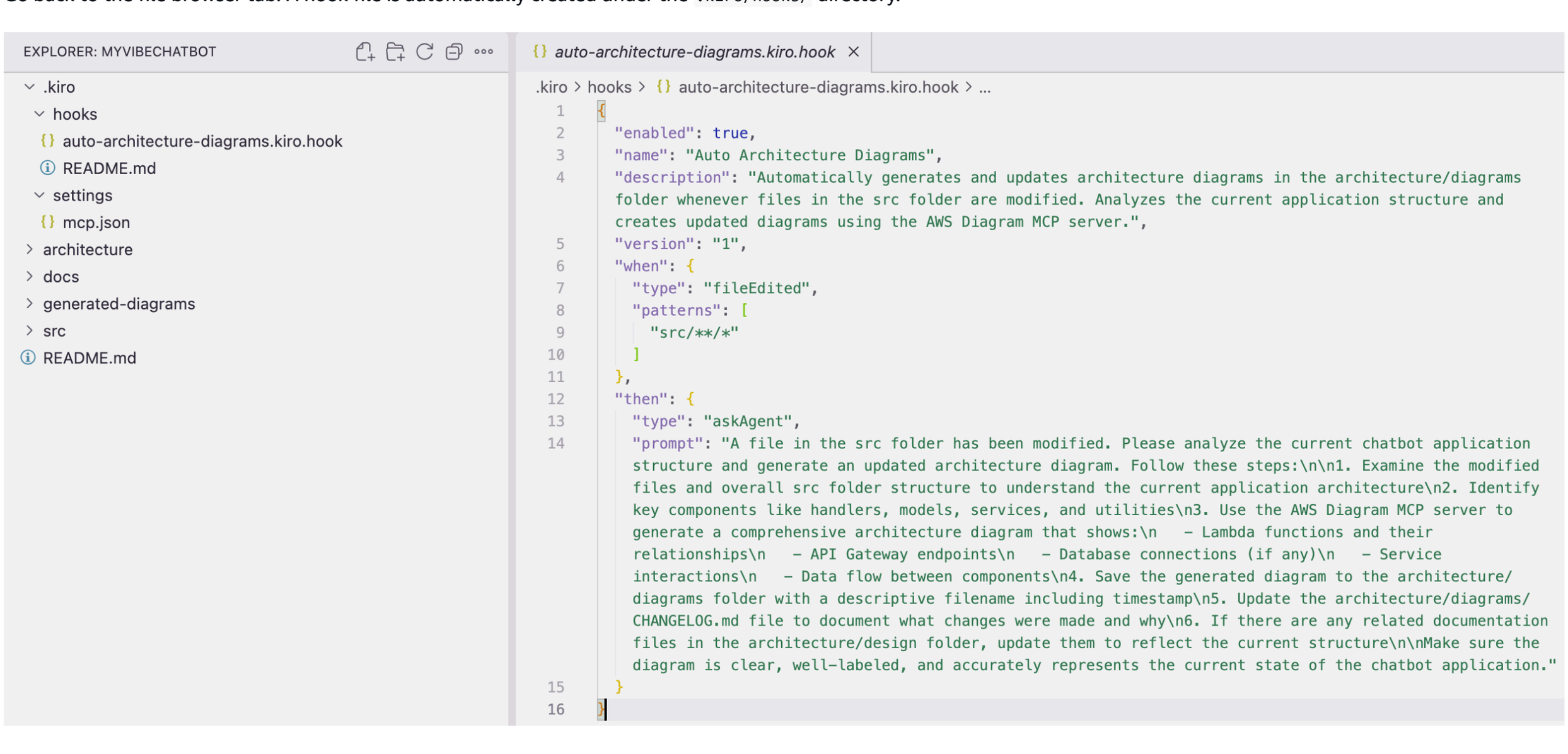
```
Please create a Kiro agent hook that will automatically generate and update architecture diagrams in the architecture/diagrams folder whenever I make changes to my c

1. Trigger when I save files in the src folder
2. Analyze the current application structure
3. Generate an updated architecture diagram using the aws-architecture-diagram skill
4. Save the diagram in the architecture/diagrams folder
5. Update any related documentation

Make sure the hook is well-documented and easy to modify. Make sure the hook is enabled.
```



Kiro creates a hook called `Auto-Update Architecture Diagrams` successfully. The hook is enabled.



Go back to the file browser tab. A hook file is automatically created under the `.kiro/hooks/` directory.



Configure the Hook

The agent hook should be configured to:

- **Trigger**: On file save events in the `src/` directory
- **Action**: Generate architecture diagrams based on current code structure
- **Output**: Save diagrams to `architecture/diagrams/` folder
- **Documentation**: Update architecture documentation automatically

Automation Benefits
This agent hook will keep your architecture diagrams synchronized with your code changes, ensuring your documentation stays current throughout development.

Step 7: Install the Draw.io Integration Plugin

Since the `aws-architecture-diagram` skill generates diagrams in draw.io XML format, you'll want to be able to view and edit them directly inside Kiro. Install the **Draw.io Integration** plugin from the Extensions Marketplace:

1. Open the Extensions panel (`Cmd + Shift + X` on macOS / `Ctrl + Shift + X` on Windows)
2. Search for **Draw.io Integration**
3. Click **Install**

Once installed, any `.drawio` or `.drawio.svg` files in your project will open as editable diagrams directly in Kiro — no need to switch to an external application.

Why Draw.io?
The new diagram skill produces draw.io XML with official AWS4 icons, which is a more portable and editable format than the Python-generated PNG images from the old MCP server. You can export to PNG, SVG, or PDF from within the Draw.io editor if needed.

Step 8: Test Your Setup

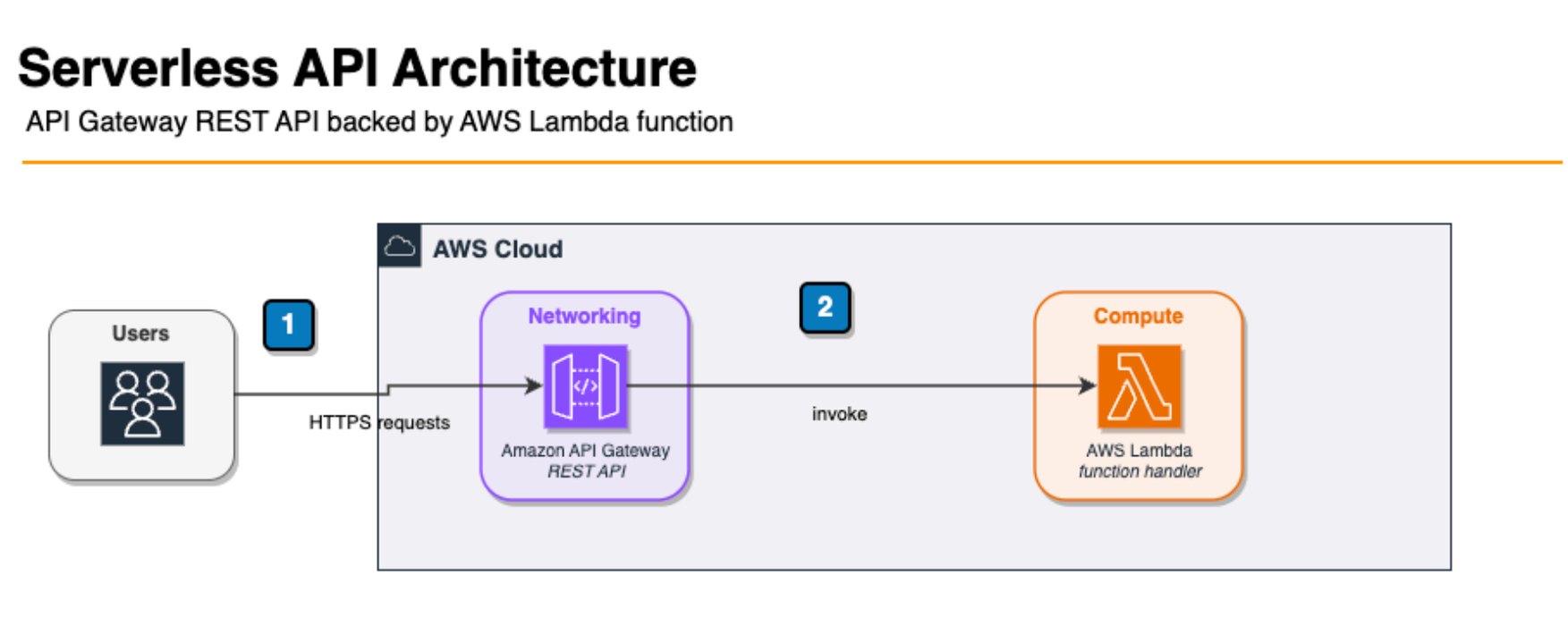
Verify everything is working correctly:

Test MCP Server and Diagram Skill

⚠️ **IMPORTANT:** If Kiro opened a new session when creating the hook, close that session once the hook is created successfully and go back to the previous session and type:

```
Can you test the AWS Documentation MCP server by looking up information about AWS Lambda? Also use the aws-architecture-diagram skill to create a simple diagram show
```

Kiro should be able to access the AWS Documentation MCP server to look up information about AWS Lambda, and a diagram like below should be created at `architecture/diagrams/lambda-api-gateway.drawio`.

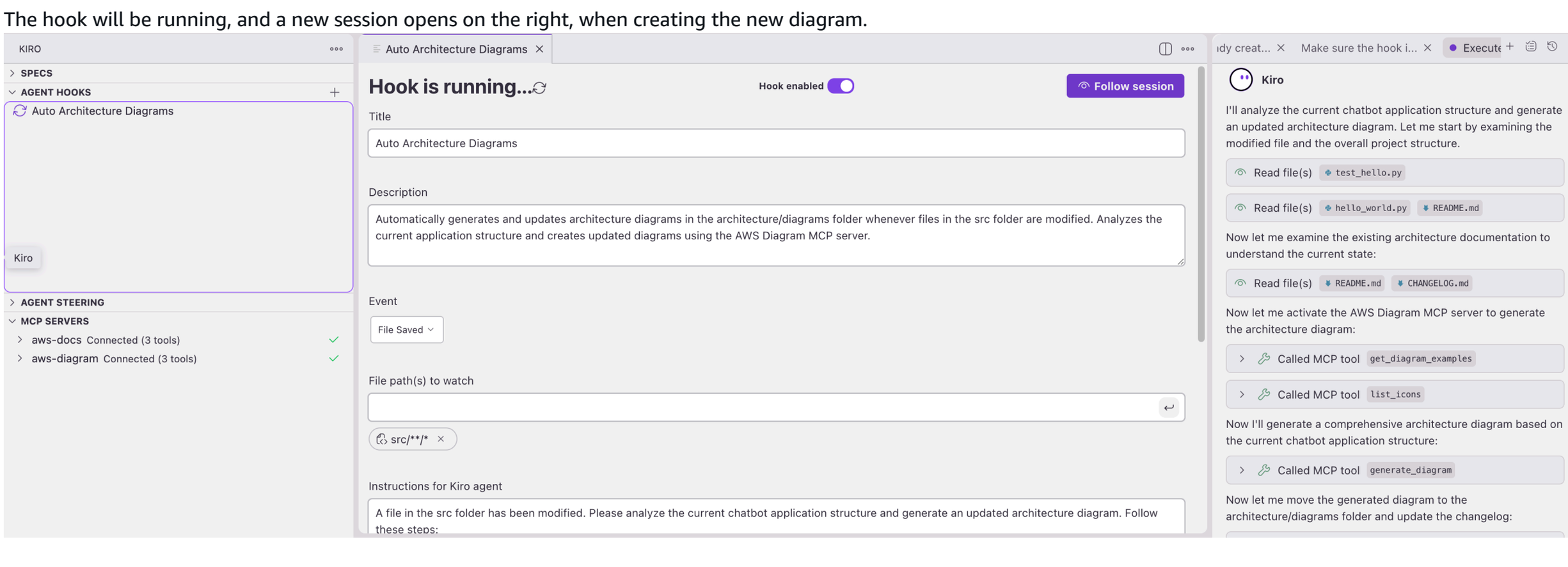


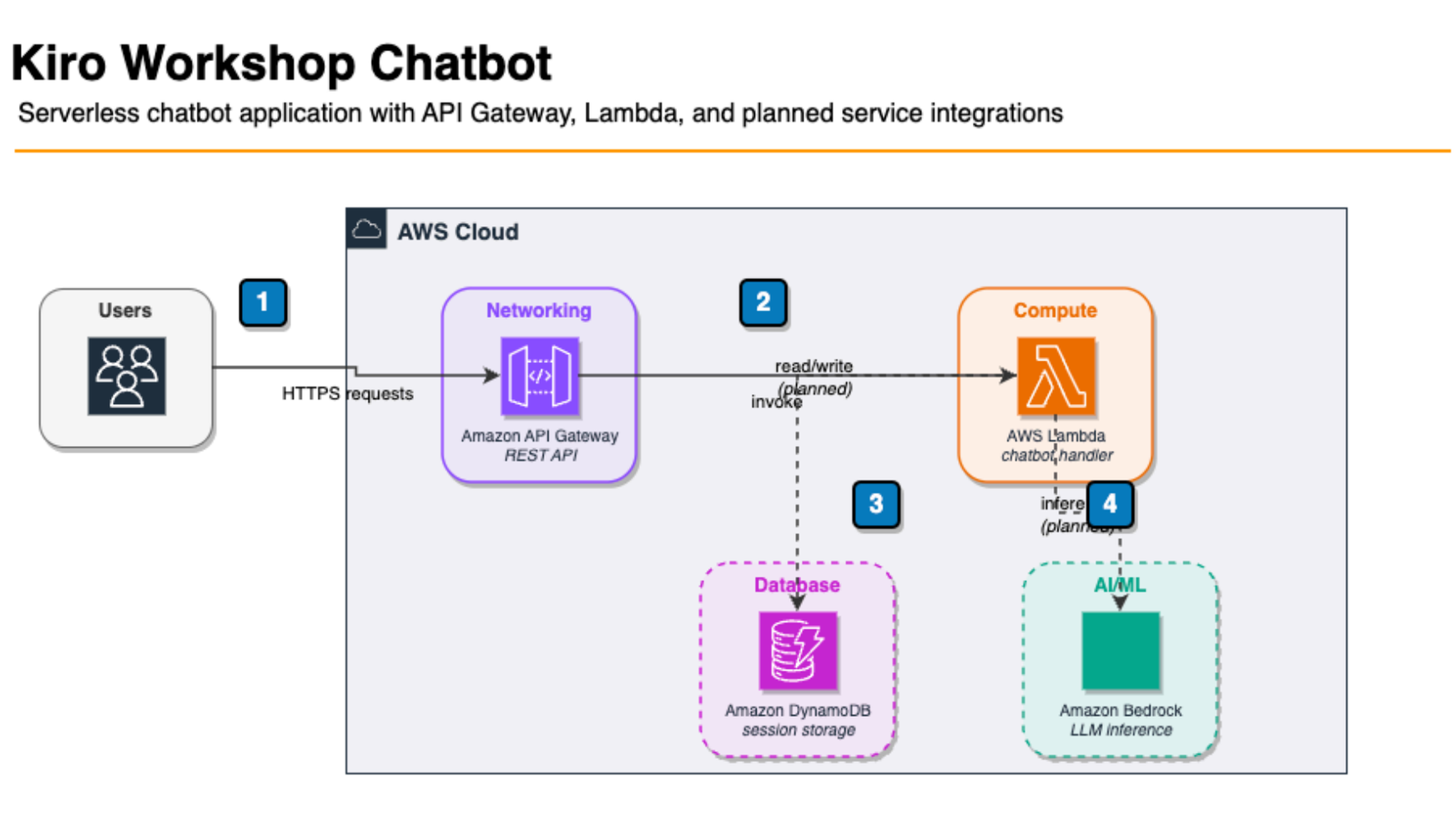
Test Agent Hook

```
Please create a simple "Hello World" file in the src folder to test if our agent hook is working correctly. The hook should automatically generate an initial archite
```

Kiro will create a Python file in `src/`. Go ahead and open the file in the editor and save it — an updated diagram should be created in the `architecture/diagrams/` directory.

The hook will be running, and a new session opens on the right, when creating the new diagram.





Expected Results

After completing this exercise, you should have:

- ✅ A properly configured Kiro project
- ✅ AWS Documentation MCP server installed and functional
- ✅ AWS architecture diagram skill installed (via deploy-on-aws agent plugin)
- ✅ Organized project folder structure
- ✅ Workshop steering document created in `.kiro/steering/`
- ✅ Automated agent hook for architecture diagram generation
- ✅ Initial architecture diagram in the `architecture/diagrams/` folder

Troubleshooting Common Issues

▸ MCP Server Installation Fails

▸ Agent Hook Not Triggering

▸ Diagrams Not Generating

Next Steps

With your development environment properly configured, you're ready to begin building your chatbot in the next exercise. The MCP server will provide AWS service information, the diagram skill will generate architecture diagrams, and the agent hook will automatically maintain your architecture documentation as you develop.

Ready for Development
Your Kiro environment is now optimized for AWS chatbot development with automated documentation and diagram generation!

Before going to the next step, instruct Kiro to clean up created source code files and generated architecture diagrams.

```
Clean up source code files and generated architecture diagrams. I would like to have a clean project to start with.
```

Previous
Next